

NPFL103: Information Retrieval (4)

Index compression, Spelling correction

Pavel Pecina

`pecina@ufal.mff.cuni.cz`

Institute of Formal and Applied Linguistics
Faculty of Mathematics and Physics
Charles University

Based on slides by Hinrich Schütze, University of Stuttgart.

Index compression

- Dictionary compression

- Postings compression

Spelling correction

- Introduction

- Edit distance

- Levenshtein distance

Index compression

Dictionary compression

- ▶ The dictionary is small compared to the postings file.
- ▶ But we want to keep it in memory.
- ▶ Also: competition with other applications, cell phones, onboard computers, fast startup time
- ▶ So compressing the dictionary is important.

Recall: Dictionary as array of fixed-width entries

Dictionary:

term	document frequency	pointer to postings list
a	656,265	→
aachen	65	→
...	...	...
zulu	221	→

space needed:

20 bytes

4 bytes

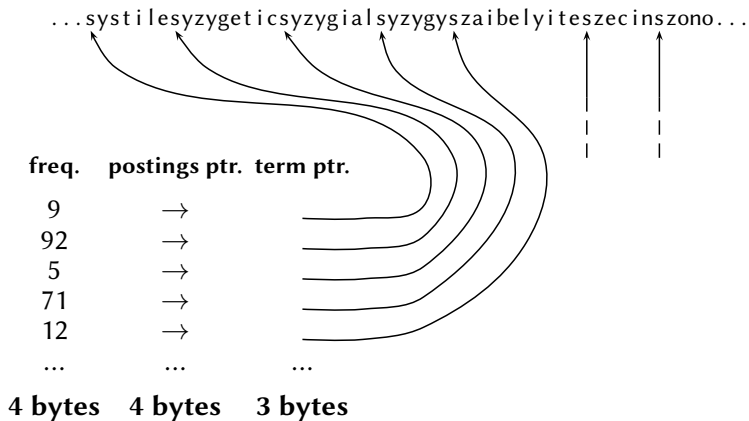
4 bytes

Space for Reuters: $(20+4+4) \times 400,000 = 11.2 \text{ MB}$

Fixed-width entries are bad

- ▶ Most of the bytes in the term column are wasted.
 - ▶ We allot 20 bytes for terms of length 1.
- ▶ We can't handle HYDROCHLOROFLUOROCARBONS and SUPERCALIFRAGILISTICEXPIALIDOCIOUS
- ▶ Average length of a term in English: 8 characters
- ▶ How can we use on average 8 characters per term?

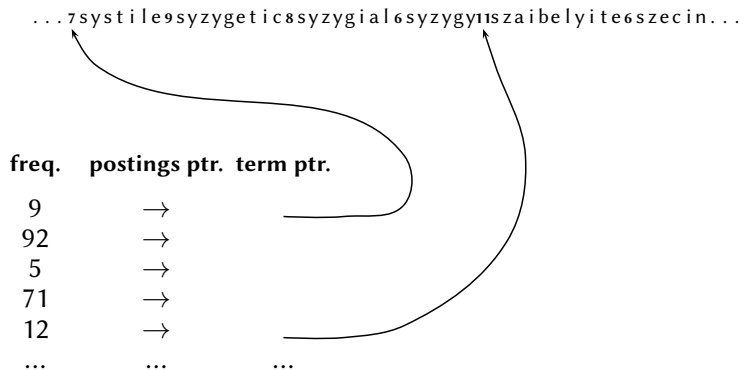
Dictionary as a string



Space for dictionary as a string

- ▶ 4 bytes per term for frequency
- ▶ 4 bytes per term for pointer to postings list
- ▶ 8 bytes (on average) for term in string
- ▶ 3 bytes per pointer into string
(need $\log_2 8 \cdot 400000 < 24$ bits to resolve $8 \cdot 400,000$ positions)
- ▶ Space: $400,000 \times (4 + 4 + 3 + 8) = 7.6\text{MB}$ (compared to 11.2 MB for fixed-width array)

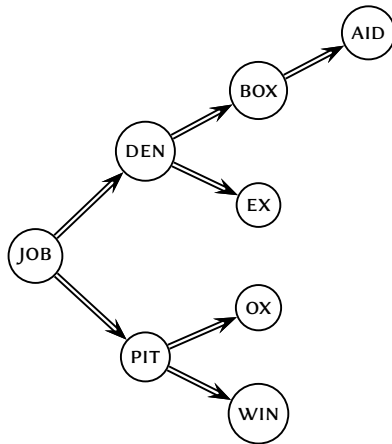
Dictionary as a string with blocking



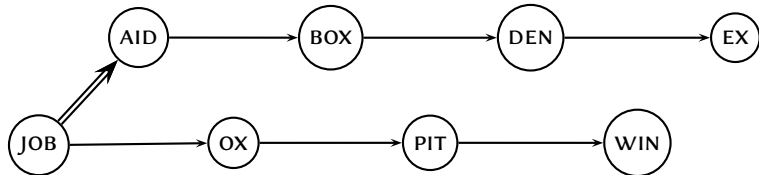
Space for dictionary as a string with blocking

- ▶ Example block size $k = 4$
- ▶ Where we used 4×3 bytes for term pointers without blocking ...
- ▶ ...we now use 3 bytes for one pointer plus 4 bytes for indicating the length of each term.
- ▶ We save $12 - (3 + 4) = 5$ bytes per block.
- ▶ Total savings: $400,000 / 4 * 5 = 0.5$ MB
- ▶ This reduces the size of the dictionary from 7.6 MB to 7.1 MB.

Lookup of a term without blocking



Lookup of a term with blocking: (slightly) slower



Front coding

One block in blocked compression ($k = 4$) ...

8 a u t o m a t a **8** a u t o m a t e **9** a u t o m a t i c **10** a u t o m a t i o n



...further compressed with front coding.

8 a u t o m a t * a **1** ◇ e **2** ◇ i c **3** ◇ i o n

Dictionary compression for Reuters: Summary

data structure	size in MB
dictionary, fixed-width	11.2
dictionary, term pointers into string	7.6
~, with blocking, $k = 4$	7.1
~, with blocking & front coding	5.9

Postings compression

- ▶ The postings file is much larger than the dictionary (factor >10)
- ▶ Key desideratum: store each posting compactly
- ▶ A posting for our purposes is a docID.
- ▶ For Reuters (800,000 documents), we would use 32 bits per docID when using 4-byte integers.
- ▶ Alternatively, we can use $\log_2 800,000 \approx 19.6 < 20$ bits per docID.
- ▶ Our goal: use a lot less than 20 bits per docID.

Key idea: Store gaps instead of docIDs

- ▶ Each postings list is ordered in increasing order of docID.
- ▶ Example postings list: COMPUTER: 283154, 283159, 283202, ...
- ▶ It suffices to store **gaps**: $283159 - 283154 = 5$, $283202 - 283154 = 43$
- ▶ Example postings list using gaps : COMPUTER: 283154, 5, 43, ...
- ▶ Gaps for frequent terms are small.
- ▶ Thus: We can encode small gaps with fewer than 20 bits.

Gap encoding

encoding postings list						
THE	docIDs	...	283042	283043	283044	283045 ...
	gaps		1	1	1	...
COMPUTER	docIDs	...	283047	283154	283159	283202 ...
	gaps		107	5	43	...
ARACHNOCENTRIC	docIDs	252000	500100			
	gaps		248100			

Compression of Reuters

data structure	size in MB
dictionary, fixed-width	11.2
dictionary, term pointers into string	7.6
~, with blocking, $k = 4$	7.1
~, with blocking & front coding	5.9
collection (text, xml markup etc)	3600.0
collection (text)	960.0
T/D incidence matrix	40,000.0
postings, uncompressed (32-bit words)	400.0
postings, uncompressed (20 bits)	250.0
postings, variable byte encoded	116.0

Term-document incidence matrix

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth ...
ANTHONY	1	1	0	0	0	1
BRUTUS	1	1	0	1	0	0
CAESAR	1	1	0	1	1	1
CALPURNIA	0	1	0	0	0	0
CLEOPATRA	1	0	0	0	0	0
MERCY	1	0	1	1	1	1
WORSER	1	0	1	1	1	0
...						

Entry 1 if term occurs. e.g. CALPURNIA occurs in *Julius Caesar*.

Entry 0 if term doesn't occur. e.g. CALPURNIA doesn't occur in *The tempest*.

Summary

- ▶ We can now create an index for highly efficient Boolean retrieval that is very space efficient.
- ▶ Only 10-15% of the total size of the text in the collection.
- ▶ However, we've ignored positional and frequency information.
- ▶ For this reason, space savings are less in reality.

Spelling correction

Spelling correction

- ▶ Two principal uses:
 1. Correcting **documents** being indexed
 2. Correcting user **queries** at query time
- ▶ Two different methods for spelling correction:
 1. **Isolated word** spelling correction
 - ▶ Check each word on its own for misspelling
 - ▶ Will not catch typos resulting in correctly spelled words, e.g., *an asteroid that fell **form** the sky*
 2. **Context-sensitive** spelling correction
 - ▶ Look at surrounding words
 - ▶ Can correct *form/from* error above

Correcting documents vs. correcting queries

- ▶ We're not interested in interactive spelling correction of documents.
- ▶ In IR, we use document correction primarily for OCR'ed documents. (OCR = optical character recognition)
- ▶ The general philosophy in IR is: don't change the documents.
- ▶ Spelling errors in queries are much more frequent

Isolated word spelling correction

► Premises:

1. There is a list of “correct words” from which the correct spellings come.
2. We have a way of computing the **distance** between a misspelled word and a correct word.

► Simple algorithm: return the “correct” word that has the smallest distance to the misspelled word.

► Example: *informaton* → *information*

► For the list of correct words, we can use the vocabulary of all words that occur in our collection.

► Why is this problematic?

Alternatives to using the term vocabulary

- ▶ A standard dictionary (Webster's, OED etc.)
- ▶ An industry-specific dictionary (for specialized IR systems)
- ▶ The term vocabulary of the collection, appropriately weighted

Distance between misspelled word and “correct” word

1. Edit (Levenshtein) distance
2. Weighted edit distance
3. k -gram overlap

Edit distance

- ▶ The edit distance between string s_1 and string s_2 is the minimum number of basic operations that convert s_1 to s_2 .
- ▶ **Levenshtein**: The basic operations are **insert**, **delete**, and **replace**.
- ▶ Examples:
 - ▶ Levenshtein distance *dog-do*: 1
 - ▶ Levenshtein distance *cat-cart*: 1
 - ▶ Levenshtein distance *cat-cut*: 1
 - ▶ Levenshtein distance *cat-act*: 2
- ▶ **Damerau-Levenshtein**: **transposition** as a fourth possible operation.
- ▶ Example:
 - ▶ Damerau-Levenshtein distance *cat-act*: 1

Levenshtein distance: Computation matrix (dynamic programming)

		f	a	s	t
	0	1	2	3	4
c	1	1	2	3	4
a	2	2	1	2	3
t	3	3	2	2	2
s	4	4	3	2	3

Levenshtein distance: Algorithm

LEVENSHTEINDISTANCE(s_1, s_2)

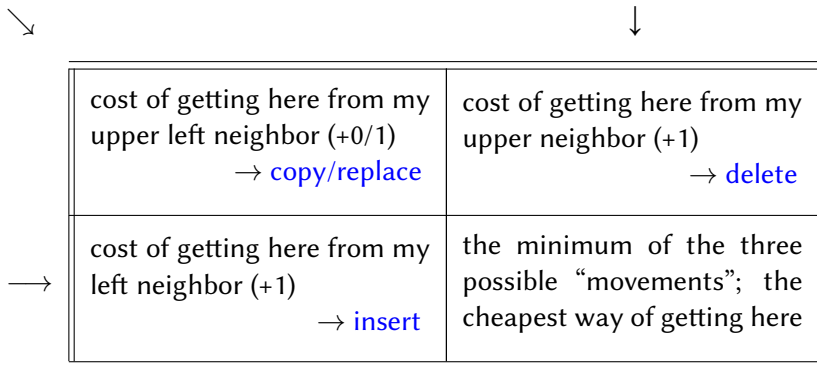
```
1  for  $i \leftarrow 0$  to  $|s_1|$ 
2  do  $m[i, 0] = i$ 
3  for  $j \leftarrow 0$  to  $|s_2|$ 
4  do  $m[0, j] = j$ 
5  for  $i \leftarrow 1$  to  $|s_1|$ 
6  do for  $j \leftarrow 1$  to  $|s_2|$ 
7      do if  $s_1[i] = s_2[j]$ 
8          then  $m[i, j] = \min\{m[i-1, j]+1, m[i, j-1]+1, m[i-1, j-1]\}$ 
9          else  $m[i, j] = \min\{m[i-1, j]+1, m[i, j-1]+1, m[i-1, j-1]+1\}$ 
10 return  $m[|s_1|, |s_2|]$ 
```

Operations: insert (cost 1), delete (cost 1), replace (cost 1), copy (cost 0)

Levenshtein distance: Example

			f		a		s		t	
		0	1	1	2	2	3	3	4	4
c		1	1	2	2	3	3	4	4	5
		1	2	1	2	2	3	3	4	4
a		2	2	2	1	3	3	4	4	5
		2	3	2	3	1	2	2	3	3
t		3	3	3	3	2	2	3	2	4
		3	4	3	4	2	3	2	3	2
s		4	4	4	4	3	2	3	3	3
		4	5	4	5	3	4	2	3	3

Each cell of Levenshtein matrix



 cost of getting here from my upper left neighbor (+0/1) → copy/replace	 cost of getting here from my upper neighbor (+1) → delete
 cost of getting here from my left neighbor (+1) → insert	the minimum of the three possible “movements”; the cheapest way of getting here

Example: Levenshtein distance OSLO – SNOW

		s	n	o	w
	0	1 1	2 2	3 3	4 4
o	1 1	2 1	2 2	3 2	3 3
s	2 2	3 1	2 2	3 3	4 3
l	3 3	4 2	3 2	3 3	4 4
o	4 4	5 3	4 3	4 2	3 3

cost	operation	input	output
1	delete	o	*
0	(copy)	s	s
1	replace	l	n
0	(copy)	o	o
1	insert	*	w

Example: Levenshtein distance CAT – CATCAT

		c	a	t	c	a	t
	0	1	2	3	4	5	6
c	1	0	2	3	3	5	6
a	2	2	1	2	3	3	5
t	3	3	2	0	2	3	3

cost	operation	input	output
1	insert	*	c
1	insert	*	a
1	insert	*	t
0	(copy)	c	c
0	(copy)	a	a
0	(copy)	t	t

Example: Levenshtein distance CAT – CATCAT

			c		a		t		c		a		t			
			0		1	1	2	2	3	3	4	4	5	5	6	6
c			1		0	2	2	3	3	4	3	5	5	6	6	7
			1		2	0	1	1	2	2	3	3	4	4	5	5
a			2		2	1	0	2	2	3	3	4	3	5	5	6
			2		3	1	2	0	1	1	2	2	3	3	4	4
t			3		3	2	2	1	0	2	2	3	3	4	3	5
			3		4	2	3	1	2	0	1	1	2	2	3	3

cost	operation	input	output
0	(copy)	c	c
1	insert	*	a
1	insert	*	t
1	insert	*	c
0	(copy)	a	a
0	(copy)	t	t

Example: Levenshtein distance CAT – CATCAT

		c	a	t	c	a	t
	0	1 1	2 2	3 3	4 4	5 5	6 6
c	1	0 2	2 3	3 4	3 5	5 6	6 7
	1	2 0	1 1	2 2	3 3	4 4	5 5
a	2	2 1	0 2	2 3	3 4	3 5	5 6
	2	3 1	2 0	1 1	2 2	3 3	4 4
t	3	3 2	2 1	0 2	2 3	3 4	3 5
	3	4 2	3 1	2 0	1 1	2 2	3 3

cost	operation	input	output
0	(copy)	c	c
0	(copy)	a	a
1	insert	*	t
1	insert	*	c
1	insert	*	a
0	(copy)	t	t

Example: Levenshtein distance CAT – CATCAT

			c		a		t		c		a		t		
			0	1	1	2	2	3	3	4	4	5	5	6	6
c			1	0	2	2	3	3	4	3	5	5	6	6	7
			1	2	0	1	1	2	2	3	3	4	4	5	5
a			2	2	1	0	2	2	3	3	4	3	5	5	6
			2	3	1	2	0	1	1	2	2	3	3	4	4
t			3	3	2	2	1	0	2	2	3	3	4	3	5
			3	4	2	3	1	2	0	1	1	2	2	3	3

cost	operation	input	output
0	(copy)	c	c
0	(copy)	a	a
0	(copy)	t	t
1	insert	*	c
1	insert	*	a
1	insert	*	t

Weighted edit distance

- ▶ As above, but operation weights depend on the characters involved.
- ▶ Meant to capture keyboard errors
(e.g., m more likely to be mistyped as n than as q).
- ▶ Therefore, replacing m by n is a smaller edit distance than by q .
- ▶ Requires a weight matrix as input.
- ▶ The dynamic programming need to be modified to handle weights.

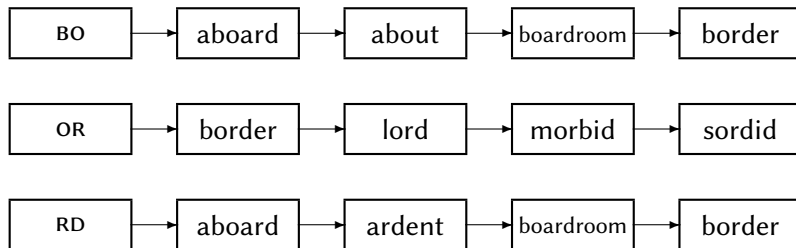
Using edit distance for spelling correction

- ▶ Given a query, first enumerate all character sequences within a preset (possibly weighted) edit distance.
- ▶ Intersect this set with our list of “correct” words.
- ▶ Then suggest terms in the intersection to the user.

k -gram indexes for spelling correction

- ▶ Enumerate all k -grams in the query term
- ▶ Example:
 - ▶ bigram index, misspelled word: *bordroom*
 - ▶ bigrams: *bo, or, rd, dr, ro, oo, om*
- ▶ Use the k -gram index to retrieve “correct” words that match query term k -grams
- ▶ Threshold by number of matching k -grams
(e.g., only vocabulary terms that differ by at most 3 k -grams)
- ▶ Rank by (weighted) edit distance

k -gram indexes for spelling correction: *bordroom*



Context-sensitive spelling correction

- ▶ Our example was: *an asteroid that fell **form** the sky*
- ▶ How can we correct *form* here?
- ▶ One idea: **hit-based** spelling correction (hit = retrieved document)
 1. Retrieve “correct” terms close to each query term
for *flew form munich*: *flea* for *flew*, *from* for *form*, *munch* for *munich*
 2. Try all possible phrases as queries with one word “fixed” at a time: “*flea form munich*”, “*flew from munich*”, “*flew form munch*”
 3. The correct query “*flew from munich*” has the most hits.
See Google’s *Did you mean?* (or *Including results for [correction]. Search only for [original].*)
- ▶ Suppose we have 7 alternatives for *flew*, 20 for *form* and 3 for *munich*, how many “corrected” phrases will we enumerate?

Context-sensitive spelling correction cont'd.

- ▶ The “hit-based” algorithm we just outlined is not very efficient.
- ▶ More efficient alternative: look at “collection” of queries ([query logs](#)), not documents.
- ▶ Another alternative: learn corrections from the users (mine query logs for sequences of a *incorrect query* followed by a *corrected query*).
- ▶ Yet another alternative: language models

General issues in spelling correction

▶ User interface:

- ▶ automatic vs. suggested correction
- ▶ *Did you mean?* only works for one suggestion.
- ▶ What about multiple possible corrections?
- ▶ Tradeoff: simple vs. powerful UI

▶ Cost:

- ▶ Spelling correction is potentially expensive.
- ▶ Avoid running on every query?
- ▶ Maybe just on queries that match few documents.
- ▶ Guess: Spelling correction of major search engines is efficient enough to be run on every query.

Index compression

- Dictionary compression

- Postings compression

Spelling correction

- Introduction

- Edit distance

- Levenshtein distance